FIKS: Turning Silent Tool Failures into Gated Repair Pull Requests for MCP Servers

System Design and Pilot Evaluation

Anvay Vats
University of Waterloo

October 2026

Author review checklist

Author review: Title, subtitle and abstract framing: confirm you want "pilot + pre-registration draft" as the paper's stated type. . . . 2
Author review: Confirm these three contributions are the ones you want to claim, in this order. . . . 3
Author review: The practitioner paragraph cites a Reddit thread. Keep it (motivation, clearly labelled) or drop it (some reviewers discount forum sources)? . . . 4
Author review: Check that §4 describes the design you intend to defend (receipt channels, contract kinds, F/P gates, sandbox limits). . . . 5
Author review: Confirm the account of who merged the pilot pull requests and how (AI assistant on your account, at your request, without line-by-line review). . . . 8
Author review: The novelty boundary is your claim to defend: patents were only partly searched (Google Patents, Espacenet and Patentscope not searched). Re-check before submission. . . . 10
Author review: Table 9 rows for commercial products come from their documentation. Spot-check the cells you will be asked about. . . . 10
Author review: The hypotheses, margins and analysis plan become binding when you tag `prereg-v1`. Read `paper/prereg/PREREG.md` and approve or change them first. . . . 10
Author review: Add any limitation you know of that is not listed here. . . . 11
Author review: Check the AI-use statement against the policy of the venue or archive you submit to. 11

Abstract

LLM agents are meant to obtain ground truth from tool results, but tools can fail silently: a Model Context Protocol (MCP) server returns a well-formed success whose content is wrong, for example after its upstream API changes behaviour without changing its schema. Published studies show that such failures occur and that agents often adopt the wrong values; a schema checks shape, not meaning, and a specification diff of an unchanged document reports nothing. Runtime monitors can warn the current agent, but the tool stays broken for every later one. We present FIKS, a loop that turns a silent tool failure into a reviewable fix. A proxy checks every successful MCP result against an outcome contract, including agreement with the raw upstream HTTP exchange and read-after-write, and returns a receipt to the agent. A violation is compiled into a deterministic reproduction test that must fail on the current code for the named property; a tool-restricted coding agent repairs the server; and a pull request is opened only after that test, the full contract suite, all stored traces, typecheck, lint and CI on the exact commit pass. FIKS never merges. In a pilot with 5 seeded drifts on a mock vendor whose OpenAPI document does not change, FIKS flagged all 5, produced a valid fail-first test for each, and obtained a green-CI pull request in one repair attempt per drift (1.4–1.6 minutes from violation to PR; \$0.11–\$0.21 per repair at API prices). In a 6-task, 5-trial agent bench, pass^5 was 50.0% without FIKS, 83.3% with receipts and 100.0% after the fixes were merged; the pilot is too small to establish the receipt effect (sign test $p = 0.50$), and an earlier receipt wording did worse than no monitor (also not significant). We position FIKS against a catalog of 508 sources, state which of its parts are not new, and release a pre-registration draft for a confirmatory study that has not yet been run.

Author review: Title, subtitle and abstract framing: confirm you want "pilot + pre-registration draft" as the paper's stated type.

Keywords: Model Context Protocol; tool-using agents; silent failures; runtime verification; API drift; reproduction tests; automated program repair

1 Introduction

When a tool fails loudly, with an exception or an error result, the agent can retry, an error tracker can file an issue, and a coding agent can be pointed at the stack trace [14, 51]. When a tool fails silently, none of this happens: a Model Context Protocol (MCP) server returns a well-formed success whose content is wrong. One cause, and the one this paper targets, is upstream drift: the vendor API behind the server changes a unit, a field name, pagination, a timestamp format or a write requirement, while its published schema stays the same; in npm, about two thirds of documented breaking changes are behavioural rather than signature changes [31]. Section 2 gives the evidence that such failures are frequently reported, that agents often adopt the wrong values and LLM judges detect false success poorly, and that monitors which catch them do not fix the tool.

FIKS turns the violation into a reviewable fix. A violation seen at the proxy already contains most of a reproduction: the exact arguments, the raw upstream exchange and the property that failed. Writing a valid reproduction test is the bottleneck of test-driven repair [23], and fail-to-pass tests can still be lax [54]. FIKS therefore (i) compiles the trace into a test that must fail on the current code *for the stated reason*, (ii) gives that test to a tool-restricted coding agent, and (iii) runs every gate, including CI on the exact commit, *before* a pull request is opened.

Contributions.

1. **A loop and its gate placement** (§4): per-call outcome contracts on successful MCP results, a receipt to the agent, trace capture with the raw upstream exchange, a named-property fail-first reproduction, gated repair, and a pull request that is never merged automatically.
2. **Gates hardened against the repair agent** (§4.4): five attacks an adversarial agent mounted on the gates, each closed and kept as a regression test, and a seed suite that rejects eight known wrong fixes.
3. **A pilot evaluation** (§5) on 5 seeded drifts, with every number regenerated by a script from committed logs, including a negative result on receipt wording.

Section 6 compares the closest systems step by step and states what is not new; Section 7 summarizes a draft protocol for the confirmatory study.

Author review: Confirm these three contributions are the ones you want to claim, in this order.

Scope and status. This paper reports a pilot: one mock vendor, one MCP server, drifts, contracts and tasks written by the author, six agent tasks and one model family. Validated here: the published evidence in §2 (each citation checked against its source; the practitioner thread was supplied by the author and is not verified), the pilot numbers (rebuilt by script from logs), and the feasibility of the loop on seeded drifts. Not validated: that FIKS outperforms a composition baseline, the effect of receipts on agent outcomes, its false-alarm rate on unseen traffic, its behaviour on real bugs and legitimate vendor changes, and maintainer acceptance of its pull requests. The draft protocol (§7) tests the composition comparison, the false-alarm rate and the behaviour on real bugs and legitimate changes; the receipt effect appears only as an optional exploratory run, and maintainer acceptance is outside the protocol.

2 Why this tool should exist: the evidence

Five lines of published evidence and one informal practitioner discussion motivate FIKS (Table 1).

Agents depend on tool results, and silent tool faults are real. Agents are meant to obtain ground truth from the environment, such as tool results, at each step [50]. In a survey of 41 practitioners, tool-response handling (malformed, incomplete or semantically unusable output after a *successful* call) was the most frequently reported MCP fault type (66.67%, ahead of documentation at 64.52%), and the authors recommend validating responses after successful invocation [55]. An analysis of 833 MCP fault threads found servers returning well-formed success responses despite incorrect behaviour [41]. The MCP reference memory server silently dropped new observations sent for existing entities (47 of 130 agent runs) [37], and an executable-contract audit found seven defects in the tools of agent benchmarks, including success messages for writes that never happened [5].

Agents do not catch them, and neither do observers. When a production API silently accepts an invalid value, agents rarely notice (12%) and never repair it; they tell the user nothing exists (41%) or invent a figure (12%) [33]. Agents adopt corrupted but plausible tool returns at high rates, up to 68% for web search [60]; implicit tool failures cause the sharpest recovery drops [66]; and agents given empty or malformed success payloads fabricated results in 56.6% of valid responses [53]. Detection from the outside is also hard: no LLM-judge configuration exceeded AUROC 0.65 at detecting false success on τ^2-bench [2], gray failure is defined by differential observability: a system's own failure detectors may not notice a problem its applications experience [25], and agent frameworks can alter valid MCP results after the wire [43].

Upstream behaviour drifts while schemas stay the same. In npm, about two thirds of documented breaking changes are behavioural rather than signature changes [31]. Real providers ship changes that clients misread without any error: GitHub treats enum additions as non-breaking [21], an LLM gateway turned a safety refusal into a normal completion through a default branch for unknown enum values [15], and a popular finance wrapper documents that its upstream sometimes returns prices 100× wrong (dollars vs cents) and ships a repair step for them [64]. Production ML practice warns that a source-system upgrade can change a feature's meaning without producing values strange enough to trigger monitoring [8]. A specification diff of an unchanged document reports nothing [40]; MCP output-schema validation is a client-side recommendation [47]; and static analysis of 1,899 MCP servers flags bug patterns but not behavioural, data-dependent faults [24].

Detection without repair does not last. Outcome monitors give the running agent recovery affordances but explicitly do not repair the tool [42]. In data validation, over-alerting leads users to ignore alerts, and an alert's value is whether it is actionable and gets fixed [45]; engineers silenced and eventually abandoned a completeness check that raised too many false alarms [52]. Error-tracking autofix starts from errors and, for automatic runs, needs recurrence (at least 10 events in 14 days), with reproduction tests only on request [51];

Table 1: Evidence that motivates FIKS and the design response to each finding.

Finding	Implication	FIKS design response
Tool-response handling after successful calls is the most frequently reported MCP fault type in a practitioner survey [55]; servers return successes despite incorrect behaviour [41]	errors and exceptions miss them	check every *successful* result
Agents rarely notice silent API failures and never repair them [33]; they adopt plausible corrupted returns [60]	the agent cannot be the detector	deterministic contracts outside the agent; receipt to the agent
LLM judges reach AUROC ≤ 0.65 on false success [2]	model-based detection is weak	compare with the raw upstream exchange (`equalsUpstream`, `readAfterWrite`)
Behavioural breaks dominate documented breaking changes in npm [31]; spec diffs of unchanged documents are empty [40]	spec-level tools are blind	per-call checks on live results
Monitors do not repair the tool [42]; noisy alerts get silenced, and alerts are judged by whether they get fixed [45, 52]	alerts must be precise and lead to a fix	turn each violation into a fix proposal
A valid reproduction test is the repair bottleneck [23]; fail-to-pass tests can be lax [54]	test validity matters	named-property fail-first test from the trace
Failed CI checks are associated with lower merge odds for agent PRs [18]	red PRs waste review	CI on the exact commit before the PR

PostHog's agent adds a regression test but does not require it to fail first, and fixes CI after opening the pull request [46].

Repair needs a valid test and green CI. Writing a true reproduction test is the bottleneck of test-driven repair [23]; fail-to-pass tests can be too lax to reject wrong patches [54] or admit partial ones [22]; and industrial agentic repair starts from an existing failing test rather than from a runtime observation [35]. On the review side, each failed CI check on an agent-authored pull request is associated with about 15% lower merge odds [18]. A runtime violation supplies the test's inputs and oracle; running CI before the pull request opens addresses the second point.

Practitioners are asking for it. In an informal Reddit discussion of benchmarks practitioners wish existed (accessed 2026-10-24) [48], several replies asked for related settings: how an agent handles a tool that returns garbage or an MCP server that dies mid-task; results the agent must decide whether to trust; writes whose response is lost or whose error claims failure although the write happened; scoring the final state of the world instead of the final answer; and agents that fail confidently while passing demos. This thread is self-selected and unverified (one respondent described itself as an AI agent), so we treat it as motivation, not as evidence of prevalence. It also marks our scope: FIKS addresses silently wrong successful results and unpersisted writes (`readAfterWrite`); it does not address servers that die mid-task, error results whose action actually succeeded, lost responses, timeouts, duplicated side effects after retries, permission drift or approvals that no longer cover an action.

The gap. In the sources we reviewed (searched up to 2026-10-21), robustness benchmarks measure how agents cope with faulty tool output [60, 66], monitors warn the agent [39, 42], and repair products start from errors [14, 51]; we found no open system or benchmark that measures whether a silent tool failure leads to a tested, reviewed fix of the tool itself (§6).

Author review: The practitioner paragraph cites a Reddit thread. Keep it (motivation, clearly labelled) or drop it (some reviewers discount forum sources)?

3 Problem definition

Let a tool τ map arguments x to a result $r = \tau(x)$ by calling an upstream service U; the call produces an upstream exchange $e = (\text{req}, \text{resp})$. A *silent failure* is a call where r is a well-formed success (no exception, no `isError`, valid against the declared `outputSchema`) but r misstates the state of U. The MCP specification lets a tool declare an `outputSchema` for its structured content, with output validation left to clients [47]; a schema can check shape, not whether a price is in dollars or cents.

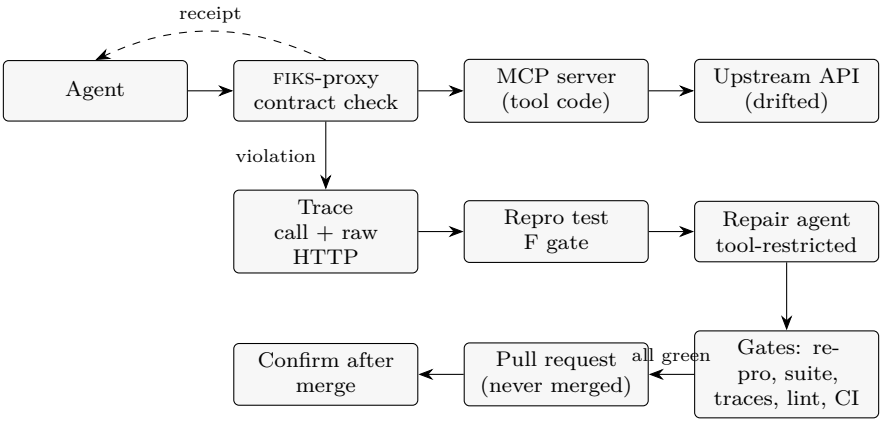


Figure 1: The FIKS loop. The receipt reaches the running agent immediately; the fix reaches every later agent once the pull request is merged (FIKS never merges).

Table 2: Contract vocabulary used in the pilot. The first five kinds are standard data-quality checks; the last two rely on seeing the upstream exchange of the same call.

Kind	Checks
`range`	numeric field within $[\min, \max]$ in a stated unit
`notNull`, `nonEmpty`	field present / list non-empty for known-good inputs
`unique`	no duplicate ids in a list
`countMatchesTotal`	returned items equal the reported total
`freshness`	timestamp within a plausible window of the call time
`equalsUpstream`	field equals the upstream response's field (JSON or instant)
`readAfterWrite`	a created resource can be read back from the upstream

We focus on *upstream behavioural drift*: U changes at time t_0 while its published specification and τ's code do not. From t_0 on, every call that touches the changed behaviour is silently wrong, for every agent, until τ is fixed. Three properties make this setting tractable for a closed loop: (a) the violation is observable per call, because the proxy sees both r and e; (b) the trace $(x, e, r, \text{property})$ is a deterministic reproduction once e is replayed; (c) the fix belongs to τ's repository, so it can be proposed as a pull request and reviewed.

4 Design of fiks

4.1 Outcome contracts and receipts

A contract is a per-tool list of invariants evaluated on each successful result (Table 2). Most kinds resemble schema and anomaly checks in data validation systems [9]; two kinds use the proxy's position between tool and upstream. `equalsUpstream` compares a result field with the field the upstream actually sent in the same call, and `readAfterWrite` re-reads a created resource from the upstream to confirm a write persisted. Each verdict is *verified*, *violated* or *unverifiable*. The proxy attaches a receipt to the result in three places: MCP `_meta`, a final text block, and a field in `structuredContent`, because some clients show the model only the structured content. A receipt states the violated property, the upstream facts the proxy observed, and what the agent should not do with the value. The receipt for the pilot's renamed-price drift reads:

```
[fiks] VIOLATED get_product: price is null, but p-1001 should have one; vendor sent unit_amount = 1199 (in
    20-2000 USD). Don't say there is no price; if you use the vendor value, say unconfirmed.
```

Author review: Check that §4 describes the design you intend to defend (receipt channels, contract kinds, F/P gates, sandbox limits).

4.2 Trace, reproduction and the F gate

On a violation the proxy stores a trace: tool, arguments, the raw upstream requests and responses recorded by an HTTP tap, the contract and the violated property ids. FIKS generates a test in the server's own repository that replays the recorded upstream exchange through the server's real code and asserts the contract. The **F gate** accepts the test only if it fails on the current code on every one of several runs (three in the current code; two in the earliest pilot runs, Table 5), and the first failure line names exactly the target property ids; a test that fails for another reason, or passes, is rejected. This makes "fails for the stated reason" a deterministic check that needs no reference patch, unlike fail-to-pass validation against a gold patch [38] or heuristic checks on the failure [3].

4.3 Repair and gates

A coding agent (Claude Agent SDK; claude-opus-5-5 in the pilot) receives the violation, the trace and the failing test in an isolated git worktree. In the pilot (prompt as first committed in `8c7daa8`, unchanged at `705e989`), the prompt also stated that the vendor had changed its behaviour without changing its published spec, included the tool's contract, each raw upstream response (truncated to 800 characters) and, when one existed, the path of the v1 recording of the same call, and gave two general notes: that responses carry a vendor version header with its dates before and after the change, and that a fix may change which requests the tool sends ("a new header, extra pages"). These notes hint at the shape of the D1, D3 and D4 fixes. A tool-permission callback lets it write only under `src/`, run only the project's typecheck, lint and test commands, and read only inside the worktree; tests, fixtures and traces are hashed at the snapshot, and any change to them fails the run. When a correct fix changes upstream requests (for example, a new required header), FIKS itself re-records the affected fixtures from the committed recordings with a frozen clock, in a separate commit containing only re-recordings. The **P gate** then requires the repro to pass, together with the full contract suite, every stored trace, typecheck and lint. FIKS pushes the branch, waits for CI on that exact commit, and opens a pull request only if CI is green; if the pull request's head differs from the commit that passed CI, it is closed. The pull request body states what broke, the trace, the test, the diff, before/after results and the gate outcomes, and says that a green suite is not a proof of correctness. FIKS never merges. After a merge, `fiks confirm` re-runs each repaired call live and replays every stored trace.

4.4 Hardening the gates against the repair agent

The repair agent is untrusted: its code runs during the gates, so it can try to change what the gates see. During development a separate agent attacked the gates as an adversary over six rounds and found five ways through: committing to bypass the write guard; having code under test rewrite FIKS's re-recordings while the gates run; a detached helper process that edited them before FIKS hashed them; steering the re-recording by changing the vendor profile it read from disk; and a planted symbolic link that redirected which committed recording was read. Each is closed, and each attack is a permanent regression test that now ends in `gate_failed` with no pull request. FIKS also disables git hooks for its own git calls, treats a refused re-recording as a failed gate, and closes any pull request whose head is not the commit that passed CI.

A second suite checks that the seed contracts accept real fixes and reject wrong ones (Table 3). It covers eight wrong fixes that an agent might plausibly write. It does not cover every wrong fix: §8 lists two that pass.

5 Pilot evaluation

5.1 Setup

Substrate. A mock vendor API with a version switch (v1, v2) and an MCP server (`shop-mcp`, four tools: `search_products`, `get_product`, `create_order`, `get_order`) in its own repository, so pull requests and CI are real. The vendor's OpenAPI document is identical in v1 and v2. Version v2 introduces 5 drifts (Table 4). The author wrote the drifts and the contracts; §8 discusses the circularity this creates.

Table 3: Seed-integrity cases (`packages/cli/test/seed-integrity.test.ts`). "Rejects" means the suite fails for the stated reason.

Case	Expected
D3 unfixed: trace fails, naming `readAfterWrite`	rejects
D3 fix without re-recording	rejects (`UNMATCHED_REQUEST`); passes after re-recording
D3 fix that also changes v1 output	rejects (v1 golden fixture)
D3: fail on the read-back instead of fixing the write	rejects
D3: cache orders inside the server instead of fixing the write	rejects
D4 fake fix (`total = items.length`)	rejects (`total.equalsUpstream`)
D4: pad items to reach the total	rejects (`items.id.unique`)
D4 real fix	passes after re-recording
D1 wrong fix (divide by 100 always)	rejects (v1 golden fixtures)
D5 lazy fix (`|| Date.now()`)	rejects (replay clock is the capture clock)
D5 real fix	passes
Turn a 404 into a made-up product	rejects (v1 golden fixture)

Table 4: Seeded behavioural drifts (vendor v2). The OpenAPI document does not change.

Drift	Upstream change	Silent symptom in the unfixed tool
D1 unit	prices become integer cents, same field name	prices 100× too high
D2 rename	`price` renamed to `unit_amount`	price returned as `null`
D3 lost write	order persists only with a new required header	create succeeds; order cannot be read back
D4 pagination	search results paginated	only page 1 returned; count < total
D5 timestamp	ISO-8601 becomes epoch milliseconds	dates parsed as 1970

Agent bench. 6 tasks (T1–T5 each exposed to one drift; T6 a control), 5 trials each, agent claude-sonnet-5-5 via the Claude Agent SDK with only the shop MCP server and a `submit_answer` tool. A deterministic grader compares the submitted answer with the vendor's ground truth; a hedged wrong value counts as wrong, and an answer never submitted counts as a failure (reported separately as "no answer"). Phases: before drift through FIKS; after drift without FIKS (agent talks to the server directly); after drift with FIKS receipts; after drift with the 5 FIKS pull requests merged. pass^k uses the unbiased $\binom{c}{k}/\binom{n}{k}$ estimator averaged over tasks [11, 63]; intervals are Wilson [10], computed over trials and therefore ignoring clustering by task, which can make them too narrow [7].

Provenance. All numbers in this section are emitted by `paper/scripts/pilot_numbers.py` from committed logs (`bench/results/`, repair records, and a snapshot of the pull requests); `-check` rebuilds them and fails on any difference. Appendix A lists the commits each run used, including uncommitted code.

5.2 Detection and false alarms

In the drift phase, 41 of 47 tool calls received a violated receipt, covering the violated properties of all five drifts (D1 `items.price.range`, D2 `price.notNull`, D3 `readAfterWrite`, D4 `countMatchesTotal`, D5 `created_at` freshness and upstream agreement). On clean v1 traffic, 0 of 37 calls were flagged (Wilson 95% upper bound 9.4% per call; 11.4% per trial over 30 trials). These bounds are weak: the pilot's clean traffic is small and comes from the same six tasks the contracts were written against.

5.3 The repair loop

Table 5 reports the loop for each drift. Each loop was triggered by a fixed one-call scenario against vendor v2, not by the bench agent's calls. All 5 reproductions passed the F gate with the named property ids (1 on three runs; 4, run on 2026-10-18, on two runs, the setting used then), all 5 repairs succeeded in one agent attempt (10–14 turns, median 12), and all 5 pull requests had green CI before opening. Total repair cost was

Table 5: Repair loop per drift. "Min. to PR" runs from the captured violation to the opened pull request, including the CI wait. Cost is the Agent SDK's API-price estimate.

Drift	Violated property	F gate (runs)	Att.	Turns	Cost ($)	Min. to PR	PR	Merged	Confirmed
D1	`items.price.range`	2/2	1	13	0.11	1.4	#2	yes	yes
D2	`price.notNull`	2/2	1	10	0.14	1.4	#1	yes	yes
D3	`readAfterWrite`	2/2	1	14	0.21	1.6	#3	yes	yes
D4	`countMatchesTotal`	3/3	1	11	0.19	1.5	#7	yes	yes
D5	`created_at.freshness, created_at.equalsUpstream`	2/2	1	12	0.12	1.4	#4	yes	yes

Table 6: Agent bench (claude-sonnet-5-5, 6 tasks × 5 trials per phase). pass[1] CI is Wilson over trials and ignores clustering by task.

Phase	pass^k (%), k=1..5					pass[1] 95% CI	Corr.	Wrong	Decl.	No ans.	Viol. calls
Before drift (v1), through fiks	100.0	100.0	100.0	100.0	100.0	89–100	30	0	0	0	0
After drift (v2), no fiks	60.0	55.0	51.7	50.0	50.0	42–75	18	5	5	2	0
After drift (v2), fiks receipts	83.3	83.3	83.3	83.3	83.3	66–93	25	0	5	0	41
After drift (v2), fiks fixes merged	100.0	100.0	100.0	100.0	100.0	89–100	30	0	0	0	0

$0.77 at API list prices; the runs used a subscription token, so this is an estimate, not a bill. After the 5 pull requests (and the harness pull request) were merged by an AI coding assistant on the owner's account at the owner's request, without line-by-line review, `fiks confirm` found each drift resolved (5/5). Across the pilot and later demonstration runs, FIKS opened 10 repair pull requests, all with green checks; 5 were merged, 4 demonstration pull requests target throw-away branches and remain open, and 1 was a duplicate opened by an operator error and closed. These merges are not evidence of maintainer acceptance, which large-scale studies of agent pull requests measure separately [18].

Author review: Confirm the account of who merged the pilot pull requests and how (AI assistant on your account, at your request, without line-by-line review).

5.4 Agent outcomes

Table 6 gives pass^k per phase and Table 7 the per-task successes. Without FIKS, pass^5 fell from 100.0% to 50.0%; with receipts it was 83.3%; with the fixes merged it returned to 100.0% with no receipts needed. Three observations temper this. First, only 2 tasks differ in the receipts-vs-no-FIKS comparison (0 in the other direction), so a task-level paired sign test gives $p = 0.50$; the pilot cannot establish an effect. Second, all 5 wrong answers without FIKS came from one task (T5, the timestamp drift: the agent reported 1970), and 2 further failures were trials where the agent stated the correct price but never submitted it; counting those as correct would raise the no-FIKS pass^5 to 66.7%, about half the gap. Third, the 5 declines with receipts are all the lost-write task T3, where the vendor really did not persist the order; "could not confirm" is arguably the right answer there, the grader counts it as a failure, and the same 5 declines occur without FIKS. Counting those declines as correct gives pass^5 of 100.0% with receipts and 66.7% without.

What these numbers can support. The unit of analysis is the task, and there are 6. Almost every task–phase cell is 5/5 or 0/5 (Table 7), so the per-trial intervals in Table 6 overstate the precision. The two runs without FIKS, on different days, differ by 16.7 points of pass^5. With 6 tasks, only a 6–0 split reaches $p < 0.05$ in a two-sided sign test ($p = 0.031$), so the agent bench can show direction and failure modes, not an effect size. Because v2 receipts include the upstream value the tool dropped, part of their benefit may come from handing the agent the correct value rather than from the warning; an arm whose receipts omit that value would separate the two.

5.5 A negative result: receipt wording

Receipts are part of the method. Our first wording (v1) stated the violation and told the agent not to quote the flagged value; apart from the timestamp drift, whose receipt is unchanged in v2, it neither gave the upstream value the tool had dropped nor suggested a next step other than telling the user the value was

Table 7: Per-task successes out of 5. T1–T5 are each exposed to one drift (D2, D1, D3, D4, D5); T6 is a control.

Phase	T1	T2	T3	T4	T5	T6
Before drift (v1), through fiks	5/5	5/5	5/5	5/5	5/5	5/5
After drift (v2), no fiks	3/5	5/5	0/5	5/5	0/5	5/5
After drift (v2), fiks receipts	5/5	5/5	0/5	5/5	5/5	5/5
After drift (v2), fiks fixes merged	5/5	5/5	5/5	5/5	5/5	5/5

Table 8: Receipt wording history on the drifted vendor (same tasks and model). v1 lowered success below no monitor.

Arm (drifted vendor)	pass[1]	pass[5]	Wrong	Declined	No answer
No fiks (2026-10-18 run)	66.7	66.7	5	5	0
fiks receipts, wording v1	43.3	33.3	0	15	2
No fiks (2026-10-20 run)	60.0	50.0	5	5	2
fiks receipts, wording v2	83.3	83.3	0	5	0

unavailable or unconfirmed. With it, pass^5 was 33.3% against 66.7% without FIKS on the same day (Table 8; 3 tasks worse, 1 better, sign test $p = 0.63$): the agent declined 15 times (all trials of T1, T2 and T3; the T3 declines also occur without FIKS) and 2 times gave no answer. The second wording (v2) adds the upstream value the tool dropped, labelled unconfirmed, and next steps. We changed the wording after seeing these results, so the v2 numbers are post hoc and need the independent test in §7.

6 Related work and novelty boundary

Silent failures and agent benchmarks. MCP fault studies classify silent faults from issue threads but do not reproduce or repair them [41]. The robustness studies in §2 evaluate how the agent responds to bad tool output [53, 60, 66], not how the tool gets fixed, and τ-bench's tools are deterministic functions over JSON databases [63].

Runtime monitors. Outcome Monitors check results and give the agent recovery affordances, explicitly without repairing the tool [42]; verimcp re-derives outcomes after git, file and CI calls [39]; a tool wrapper combining postcondition verification, verify-before-retry and idempotency keys reduced duplicate actions in simulation [36]. Action-side enforcement such as AgentSpec checks planned actions rather than results [57]. FIKS's receipt step belongs to this line and is not new on its own.

Specification, contract and data validation. Consumer-driven contracts need the provider to run consumer pacts [20]; schema-derived fuzzing finds schema violations [65]. Large-scale studies count breaking changes in library APIs [31, 58]. Data validation systems enforce similar presence, range and distribution checks on batches [9], and invariant inference goes back to Daikon [19]. Record/replay tools turn captured traffic into regression tests whose oracle is the old response [30]; that oracle encodes the bug when the recording is made after the drift.

Reproduction and repair. Issue-to-test generation is hard and its validity is judged by fail-to-pass execution [3, 29, 38]. In a self-healing agent harness 55% of gate-rejected instruction changes improved their trigger while regressing a protected case [56]. Repair systems start from issues [59, 61], compile errors after a dependency bump [49], or an upstream breaking change with no localizing failing test [62]. Structured upstream evidence helps dependency repair while raw evidence hurts [62]; FIKS gives the agent a failing test built from the trace, although the pilot prompt also included the raw upstream responses (§4).

Table 9: Loop steps covered by the closest systems (Y documented; P partial or different form; N not done or not described, and absence from documentation is not proof; ? documentation silent on the step). Sem: semantic check of a successful result; Rcpt: verdict to the agent; Trace: raw upstream captured; F: test must fail first for the stated reason; Gates: suite and CI pass before the PR.

System	Sem	Rcpt	Trace	Repro test	F	Fix	Gates	PR	Confirm
FIKS (this paper)	Y	Y	Y	Y	Y	Y	Y	Y	Y
Outcome Monitors [42]	Y	Y	N	N	N	N	N	N	N
verimcp [39]	Y	Y	P	N	N	N	N	N	N
AegisFlow [4]	Y	N	P	N	N	Y	P	N	Y
LangSmith Engine [32]	P	N	P	N	P	Y	N	Y	Y
Cleric [13]	P	N	P	N	N	Y	?	Y	Y
PostHog Self-driving [46]	N	N	P	Y	?	Y	N	Y	?
Sentry Seer [51]	N	N	P	P	N	Y	N	Y	N
Keploy [30]	N	N	Y	Y	N	N	N	N	N
SapFix [27]	N	N	P	Y	P	Y	P	Y	N
Itzal [17]	P	N	P	N	P	Y	P	P	N

Production-driven and commercial loops. ClearView patched deployed software from learned invariants [44]; Itzal validated production-driven patches on traffic [17]; SapFix shipped detect–test–repair with human approval [27]. In 2026, observability and alerting products open fix pull requests from errors, incidents or LLM scans and iterate CI after opening [14, 32, 46, 51], and some measure recurrence after fixes [13]. AegisFlow detects silent pipeline drift statistically and deploys validated patches autonomously for pre-approved failure classes [4]. A granted patent claims language-model repair of a detected issue from its error information, with dependent claims adding a pull request accompanied by a software test [1].

What is and is not new. Table 9 compares the closest systems by loop step, from their documentation or papers. Each step exists somewhere. In the sources we reviewed (searched up to 2026-10-21; patent databases only partly searched), we found no system that combines: a trigger that is a deterministic contract violation in a *successful* tool result, checked per call against the raw upstream exchange; a committed test that must fail first for the named property; and all gates, including CI on the exact commit, before the pull request opens, with a receipt to the agent while the fix is pending. We claim only that combination, and the pilot does not yet show that it outperforms a composition of existing parts; that is the question of §7.

Author review: The novelty boundary is your claim to defend: patents were only partly searched (Google Patents, Espacenet and Patentscope not searched). Re-check before submission.

Author review: Table 9 rows for commercial products come from their documentation. Spot-check the cells you will be asked about.

7 Planned confirmatory study

A draft protocol is in the repository (`paper/prereg/PREREG.md`); it becomes binding only when tagged `prereg-v1`, and none of its runs has been performed. It compares FIKS with two alternatives on the same monitor, model, budget and sandbox: **L1**, the same agent and gates with a pre-PR replay of the triggering trace but no visible reproduction test, and **COMP**, monitor → alert → coding agent → PR → CI with source-run replay. Four hypotheses form the confirmatory family (Holm, FWER 0.05): **H1**, fewer than 2% of at least 1,500 blind-authored catalog entities give rise to any false positive; **H2**, the trace-derived repro separates correct from incorrect patches better than an agent-written fail-first repro; **H3/H4**, FIKS exposes fewer green-but-incorrect pull requests than L1 and than COMP, with a correct-PR loss no larger than 0.10. The draft's own power analysis states that if the true correct-PR cost is 0.03–0.05 or the true harm gap is 0.05, H3 and H4 are more likely than not to come out "not established". Drifts are authored blind to the contracts and applied by a third-party injector; benign evolution, frozen mutation operators, four real silent-failure bugs in public MCP servers and an adversarial-trace suite are exploratory arms.

Author review: The hypotheses, margins and analysis plan become binding when you tag `prereg-v1`. Read `paper/prereg/PREREG.md` and approve or change them first.

8 Threats to validity and limitations

Circularity. The author wrote the drifts, the contracts and the tasks; detection of D1–D5 is therefore close to a construction property. Mutation-based oracle assessment and real faults are needed to measure what contracts miss [26, 28], and runtime contract vocabularies have known blind spots on out-of-vocabulary faults [42].

Scale and statistics. 6 tasks and 5 trials cannot support task-level significance; trial-level intervals ignore clustering [7]. The false-alarm bound comes from 37 clean calls.

Partial or overfitted patches. A green suite shows the repro and the recorded contracts pass; it does not prove correctness [22, 54]. During development, an adversarial verifier showed that two wrong fixes pass the test suite: a D2 fix that returns a constant in-range price, and a D1 fix that clamps prices to the range ceiling. The pilot measured no held-out oracle.

Prompt hints. The pilot repair prompt (§4) named the failure class, included the contract and the raw upstream responses, and its general notes point toward the D1, D3 and D4 fixes; the pilot fixes are small, and one-attempt success on these drifts may not carry over to repairs without such context.

Security. The repair agent reads attacker-controllable vendor bytes and its code runs in tests. Restricting terminal access does not stop injected agents from writing malicious source [34], test-passing patches can hide vulnerabilities [12], and injection through tool output is measurable [6, 16]. In the pilot the agent ran without an OS or network sandbox; a container backend and an injection suite are part of the protocol, not of these results.

Legitimate evolution. A fixed contract cannot tell a vendor fault from an intended change; the pilot always proposed a code fix. A contract-update outcome is designed but unevaluated.

Pilot build differences. 4 of 5 F gates ran twice rather than three times; all 5 repairs and the receipt-v1 runs used uncommitted code; and every repair was triggered by a scripted call, not by agent traffic.

External validity. One mock vendor, one server, one model family (Claude) for both bench and repair, and seeded rather than organic drift. Merged pull requests were merged by an AI assistant at the owner's request without line-by-line review, so they say nothing about maintainer acceptance.

Author review: Add any limitation you know of that is not listed here.

9 AI use and ethics statement

The system, the experiments and this manuscript were produced with substantial assistance from Claude (Claude Code and the Claude Agent SDK; claude-opus-5-5 and claude-sonnet-5-5), directed by the author. Literature search and source verification were done by agents that recorded the supporting passage for each citation, except the author-supplied practitioner thread and the Anthropic engineering post (claim C1 in RESEARCH.md); every number in §5 is generated from committed logs by a script. The author is responsible for all claims. No third-party repository was written to: pull requests were opened only on repositories owned by the author, and real-bug reproductions use private clones.

Author review: Check the AI-use statement against the policy of the venue or archive you submit to.

10 Conclusion

FIKS shows that a silent tool failure caught by the proxy on a call against a drifted vendor can be turned, within minutes, into a named-property failing test and a gate-passing pull request against the tool. The

pilot demonstrates feasibility on seeded drifts; it does not show that the loop beats a composition of existing monitors and coding agents, that its contracts generalize beyond the drifts they were written for, or that its pull requests would be accepted. The draft protocol is designed to answer the first two questions; maintainer acceptance needs a separate study.

References

[1] Rajeev Prabhu Acharya, Siamak Ahari, and LLC) Vinay Rao (assignee Microsoft Technology Licensing. Detecting and repairing reliability issues in operating systems and applications using a generative artificial intelligence system (us 12,711,009 b2). USPTO (granted 18 Aug 2026; pre-grant pub. US 2025/0045148 A1; provisional 31 Jul 2023), 2026. URL https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/12711009. Accessed 2026-10-21.

[2] Laksh Advani. From confident closing to silent failure: Characterizing false success in llm agents. arXiv:2606.09863, 2026. URL https://arxiv.org/abs/2606.09863. Accessed 2026-10-21.

[3] Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. Otter: Generating tests from issues to validate swe patches. arXiv:2502.05368, 2025. URL https://arxiv.org/abs/2502.05368. Accessed 2026-10-21.

[4] Muhammad Bilal Awan, Zubair Hussain, and Abdul Shahid. Aegisflow: A multi-agent agentic ai framework for autonomous remediation and self-healing in fragile data ecosystems. arXiv:2610.06971, 2026. URL https://arxiv.org/abs/2610.06971. Accessed 2026-10-21.

[5] Rohith Reddy Bellibatlu, Zichong Wang, and Wenbin Zhang. Do agent benchmarks do what they say? an executable-contract audit of tool-using agent environments. arXiv:2609.37315, 2026. URL https://arxiv.org/abs/2609.37315. Accessed 2026-10-21.

[6] Dimitrios Stamatios Bouras, Yihan Dai, and Sergey Mechtaev. Authority is not a string: A capability-scoped harness for prompt-injection-resistant coding agents. arXiv:2609.08371, 2026. URL https://arxiv.org/abs/2609.08371. Accessed 2026-10-21.

[7] Sam Bowyer, Laurence Aitchison, and Desi R. Ivanova. Position: Don't use the clt in llm evals with fewer than a few hundred datapoints. arXiv:2503.01747, 2025. URL https://arxiv.org/abs/2503.01747. Accessed 2026-10-21.

[8] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. The ml test score: A rubric for ml production readiness and technical debt reduction. IEEE International Conference on Big Data 2017, 2017. URL https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/. Accessed 2026-10-21.

[9] Eric Breck, Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. Data validation for machine learning. SysML 2019 (Google Research), 2019. URL https://research.google/pubs/data-validation-for-machine-learning/. Accessed 2026-10-21.

[10] Lawrence D. Brown, T. Tony Cai, and Anirban DasGupta. Interval estimation for a binomial proportion. Statistical Science 16(2):101-133, 2001. URL https://doi.org/10.1214/ss/1009213286. Accessed 2026-10-21.

[11] Mark Chen, Jerry Tworek, Heewoo Jun, and (58 authors). Evaluating large language models trained on code. arXiv:2107.03374, 2021. URL https://arxiv.org/abs/2107.03374. Accessed 2026-10-21.

[12] Simin Chen, Yixin He, Suman Jana, and Baishakhi Ray. Red teaming program repair agents: When correct patches can hide vulnerabilities. arXiv:2509.25894, 2025. URL https://arxiv.org/abs/2509.25894. Accessed 2026-10-21.

[13] Peter Richens (Cleric). How we verify cleric's production fixes. cleric.ai blog (2026-07-22), 2026. URL https://cleric.ai/blog/verifying-fixes. Accessed 2026-10-21.

[14] Cognition. Devin oncall. docs.devin.ai, 2026. URL https://docs.devin.ai/product-guides/devin-oncall. Accessed 2026-10-21.

[15] LiteLLM contributors. fix(core): map anthropic 'refusal' finish reason to 'content_filter' (berriai/litellm #23899; issue #23793). GitHub, 2026. URL https://github.com/BerriAI/litellm/pull/23899. Accessed 2026-10-21.

[16] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. arXiv:2406.13352, 2024. URL https://arxiv.org/abs/2406.13352. Accessed 2026-10-21.

[17] Thomas Durieux, Youssef Hamadi, and Martin Monperrus. Production-driven patch generation and validation. arXiv:1609.06848, 2016 (v2 2018). URL https://arxiv.org/abs/1609.06848. Accessed 2026-10-21.

[18] Ramtin Ehsani, Sakshi Pathak, Shriya Rawal, Abdullah Al Mujahid, Mia Mohammad Imran, and Preetha Chatterjee. Where do ai coding agents fail? an empirical study of failed agentic pull requests in github. arXiv:2601.15195, 2026. URL https://arxiv.org/abs/2601.15195. Accessed 2026-10-21.

[19] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The daikon system for dynamic detection of likely invariants. Science of Computer Programming 69(1-3):35-45, Dec 2007, 2007. URL https://homes.cs.washington.edu/~mernst/pubs/daikon-tool-scp2007-abstract.html. Accessed 2026-10-21.

[20] Pact Foundation. How pact works. docs.pact.io, 2026. URL https://docs.pact.io/getting_started/how_pact_works. Accessed 2026-10-21.

[21] GitHub. Breaking changes (github rest api docs). GitHub Docs, 2026. URL https://docs.github.com/en/rest/about-the-rest-api/breaking-changes. Accessed 2026-10-21.

[22] Yaoqi Guo, Yang Liu, Jie M. Zhang, Yun Ma, Yiling Lou, and Zhenpeng Chen. Swe-doctor: Guiding software engineering agents with runtime diagnosis from multi-faceted bug reproduction tests. arXiv:2607.00990, 2026. URL https://arxiv.org/abs/2607.00990. Accessed 2026-10-21.

[23] Kevin Han, Siddharth Maddikayala, Tim Knappe, Om Patel, Austen Liao, and Amir Barati Farimani. Tdflow: Agentic workflows for test driven development. arXiv:2510.23761, 2025. URL https://arxiv.org/abs/2510.23761. Accessed 2026-10-21.

[24] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers. arXiv:2506.13538, 2025. URL https://arxiv.org/abs/2506.13538. Accessed 2026-10-21.

[25] Peng (Ryan) Huang, Chuanxiong Guo, Lidong Zhou, Jacob R. Lorch, Yingnong Dang, Murali Chintalapati, and Randolph Yao. Gray failure: The achilles' heel of cloud-scale systems. HotOS '17 (ACM), pp. 150-155, 2017. URL https://www.microsoft.com/en-us/research/publication/gray-failure-achilles-heel-cloud-scale-systems/. Accessed 2026-10-21.

[26] Gunel Jahangirova, David Clark, Mark Harman, and Paolo Tonella. Test oracle assessment and improvement. ISSTA 2016, pp. 247-258, 2016. URL https://discovery.ucl.ac.uk/1493269/1/main.pdf. Accessed 2026-10-21.

[27] Yue Jia, Ke Mao, and Mark Harman. Finding and fixing software bugs automatically with sapfix and sapienz (meta engineering blog). Meta Engineering blog, 13 Sep 2018, 2018. URL https://engineering.fb.com/2018/09/13/developer-tools/finding-and-fixing-software-bugs-automatically-with-sapfix-and-sapienz/. Accessed 2026-10-21.

[28] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? FSE 2014, pp. 654-665, 2014. URL https://homes.cs.washington.edu/~rjust/publ/mutants_real_faults_fse_2014.pdf. Accessed 2026-10-21.

[29] Sungmin Kang, Juyeon Yoon, and Shin Yoo. Large language models are few-shot testers: Exploring llm-based general bug reproduction. arXiv:2209.11515, 2023. URL https://arxiv.org/abs/2209.11515. Accessed 2026-10-21.

[30] Keploy. What is keploy? Keploy Docs; GitHub keploy/keploy (Apache-2.0), 2026. URL https://keploy.io/docs/concepts/what-is-keploy. Accessed 2026-10-21.

[31] Dezhen Kong, Jiakun Liu, Lingfeng Bao, and David Lo. Towards better comprehension of breaking changes in the npm ecosystem. arXiv:2408.14431, 2025. URL https://arxiv.org/html/2408.14431v2. Accessed 2026-10-21.

[32] LangChain. Langsmith cloud changelog (engine entries, sep-oct 2026). LangChain docs, 2026. URL https://docs.langchain.com/langsmith/changelog. Accessed 2026-10-21.

[33] Zongrong Li, Shengkun Ye, Feiyou Guo, and Zuoyou Dang. Silentprobe: Measuring silent failure in production apis used as agent tools. arXiv:2609.00035, 2026. URL https://arxiv.org/abs/2609.00035. Accessed 2026-10-21.

[34] Liu, Zhao, Lyu, Zhang, Wang, and Lo. "your ai, my shell": Demystifying prompt injection attacks on agentic ai coding editors. arXiv:2509.22040, 2025. URL https://arxiv.org/abs/2509.22040. Accessed 2026-10-21.

[35] Chandra Maddila, Adam Tait, and Peter C. Rigby. Agentic program repair from test failures at scale: A neuro-symbolic approach with static analysis and test execution feedback. arXiv:2507.18755, 2025. URL https://arxiv.org/abs/2507.18755. Accessed 2026-10-21.

[36] Isham Kalappurackal Mansoor, Abhishek Phadke, and Pratip Rana. Verified tool calls improve llm agent reliability under non-atomic failures. arXiv:2608.02645, 2026. URL https://arxiv.org/abs/2608.02645. Accessed 2026-10-21.

[37] modelcontextprotocol/servers issue #4887. memory: create_entities silently drops observations for existing entities. GitHub, 2026. URL https://github.com/modelcontextprotocol/servers/issues/4887. Accessed 2026-10-21.

[38] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. Swt-bench: Testing and validating real-world bug-fixes with code agents. arXiv:2406.12952, 2024. URL https://arxiv.org/abs/2406.12952. Accessed 2026-10-21.

[39] Rudra Naresh. verimcp: a trusted agent stack for mcp (pypi package and repo). PyPI / GitHub rudranaresh0201/mcp (v0.3.0, 2026-08-15), 2026. URL https://pypi.org/project/verimcp/. Accessed 2026-10-21.

[40] oasdiff maintainers (Reuven Harrison). oasdiff: Openapi diff and breaking changes (breaking-changes.md). GitHub oasdiff/oasdiff (Apache-2.0), 2026. URL https://github.com/oasdiff/oasdiff/blob/main/docs/BREAKING-CHANGES.md. Accessed 2026-10-21.

[41] Joshua Owotogbe, Indika Kumara, Antonio Ken Iannillo, Willem-Jan van den Heuvel, Damian Andrew Tamburri, and Roberto Natella. A taxonomy of runtime faults in model context protocol servers. arXiv:2606.05339, 2026. URL https://arxiv.org/html/2606.05339. Accessed 2026-10-21.

[42] Sugam Panthi and Rabab Abdelfattah. Outcome monitors: Recovery affordances for silent tool failures. arXiv:2608.19303, 2026. URL https://arxiv.org/html/2608.19303. Accessed 2026-10-21.

[43] Aditi Patodiya. Localizing post-wire semantic changes in mcp agent frameworks. arXiv:2610.00182, 2026. URL https://arxiv.org/abs/2610.00182. Accessed 2026-10-21.

[44] Jeff H. Perkins, Sunghun Kim, Sam Larsen, Saman Amarasinghe, Michael D. Ernst, and Martin Rinard. Automatically patching errors in deployed software. SOSP 2009, 2009. URL https://people.csail.mit.edu/rinard/paper/sosp09.pdf. Accessed 2026-10-21.

[45] Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. Data lifecycle challenges in production machine learning: A survey. SIGMOD Record 47(2), June 2018, 2018. URL https://sigmodrecord.org/publications/sigmodRecord/1806/pdfs/04_Surveys_Polyzotis.pdf. Accessed 2026-10-21.

[46] PostHog. Anatomy of a pull request (posthog self-driving). PostHog docs, 2026. URL https://posthog.com/docs/self-driving/anatomy-of-a-pr. Accessed 2026-10-21.

[47] Model Context Protocol project. Model context protocol specification 2026-07-28 (latest): Tools. modelcontextprotocol.io, 2026. URL https://modelcontextprotocol.io/specification/latest/server/tools. Accessed 2026-10-21.

[48] r/AI_Agents. What benchmark do you wish someone would build? Reddit discussion thread, r/AI_Agents, 2026. URL https://www.reddit.com/r/AI_Agents/comments/1wvojcs/. Accessed 2026-10-24.

[49] Frank Reyes, May Mahmoud, Federico Bono, Sarah Nadi, Benoit Baudry, and Martin Monperrus. Byam: Fixing breaking dependency updates with large language models. arXiv:2505.07522, 2025. URL https://arxiv.org/abs/2505.07522. Accessed 2026-10-21.

[50] Erik Schluntz and Barry Zhang. Building effective agents. Anthropic Engineering, 2024. URL https://www.anthropic.com/research/building-effective-agents. Accessed 2026-10-21.

[51] Sentry. Issue fix (seer). Sentry docs, 2026. URL https://docs.sentry.io/product/ai-in-sentry/seer/issue-fix. Accessed 2026-10-21.

[52] Shreya Shankar, Labib Fawaz, Karl Gyllstrom, and Aditya G. Parameswaran. Moving fast with broken data. arXiv:2303.06094, 2023. URL https://arxiv.org/abs/2303.06094. Accessed 2026-10-21.

[53] Aarushi Singh. Guardrails as scapegoats: Auditing unfaithful safety refusals in tool-augmented llm agents. arXiv:2607.19449, 2026. URL https://arxiv.org/abs/2607.19449. Accessed 2026-10-21.

[54] Yuhao Tan, Zhibang Yang, Fangkai Yang, Yuan Yao, Yu Kang, and Lu Wang. Beyond fail-to-pass: Iterative hardening of co-generated bug reproduction tests and fixes. arXiv:2607.19843, 2026. URL https://arxiv.org/abs/2607.19843. Accessed 2026-10-21.

[55] Mina Taraghi, Mohammad Mehdi Morovati, and Foutse Khomh. Real faults in model context protocol (mcp) software: a comprehensive taxonomy. arXiv:2603.05637, 2026. URL https://arxiv.org/abs/2603.05637. Accessed 2026-10-21.

[56] Sina Tayebati, Divake Kumar, Nastaran Darabi, Ranganath Krishnan, and Amit Ranjan Trivedi. Self-healing harness for runtime oversight of agent self-modification. arXiv:2609.24130, 2026. URL https://arxiv.org/abs/2609.24130. Accessed 2026-10-21.

[57] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. arXiv:2503.18666, 2025. URL https://arxiv.org/abs/2503.18666. Accessed 2026-10-21.

[58] Laerte Xavier, Aline Brito, Andre Hora, and Marco Tulio Valente. Historical and impact analysis of api breaking changes: A large-scale study. SANER 2017, Klagenfurt (IEEE), 2017. URL https://homepages.dcc.ufmg.br/~mtov/pub/2017-saner-breaking-apis.pdf. Accessed 2026-10-21.

[59] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. arXiv:2407.01489, 2024. URL https://arxiv.org/abs/2407.01489. Accessed 2026-10-21.

[60] Hoyeol Yang, Woojung Song, Taewon Kim, Jonghyun Song, Seoyeon Park, and Yohan Jo. Agents' overreliance on unreliable tools. arXiv:2609.05587, 2026. URL https://arxiv.org/abs/2609.05587. Accessed 2026-10-21.

[61] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. arXiv:2405.15793, 2024. URL https://arxiv.org/abs/2405.15793. Accessed 2026-10-21.

[62] Shenghao Yang, Bo Lu, Yao Liu, Yu Kang, Qiongfang Zhang, Chetan Bansal, and Saravan Rajmohan. Deprepair: Llm-based source-code repair for dependency breaking changes. arXiv:2607.17957, 2026. URL https://arxiv.org/abs/2607.17957. Accessed 2026-10-21.

[63] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045, 2024. URL https://arxiv.org/abs/2406.12045. Accessed 2026-10-21.

[64] yfinance maintainers. Price repair (yfinance docs). ranaroussi.github.io/yfinance, 2026. URL https://ranaroussi.github.io/yfinance/advanced/price_repair.html. Accessed 2026-10-21.

[65] Dmitry Dygalo Zac Hatfield-Dodds. Deriving semantics-aware fuzzers from web api schemas. arXiv:2112.10328, 2022. URL https://arxiv.org/abs/2112.10328. Accessed 2026-10-21.

[66] Dongsheng Zhu, Xuchen Ma, Yucheng Shen, Xiang Li, Yukun Zhao, Shuaiqiang Wang, Lingyong Yan, and Dawei Yin. When tools fail: Benchmarking dynamic replanning and anomaly recovery in llm agents (toolmaze). arXiv:2606.05806, 2026. URL https://arxiv.org/abs/2606.05806. Accessed 2026-10-21.

A Reproducing the pilot numbers

Code, logs and the sources of this manuscript are public at https://github.com/vats98754/fiks; the mock server is at https://github.com/vats98754/shop-mcp. The first four commands below need only Python and Node.js; the live demo also needs a Claude Code OAuth token and GitHub CLI credentials.

```
python3 paper/scripts/pilot_numbers.py --check # rebuild every macro and table from bench/results
python3 paper/scripts/make_bib.py --check # references.bib from verified sources only
pnpm fiks summarize --check # bench summary.json matches the trial logs
pnpm fiks vendor-diff # OpenAPI identical across v1/v2; five behaviours differ
pnpm demo --drifts D2 --base fresh # one live loop: violation -> repro -> repair -> green CI -> PR
```

Run provenance. Phases A, B and B-direct used FIKS commit `705e989` and the server at tag `seed-v1` (`bb037cf`); phase C used the server's main branch at `7bf2e66` (the 5 fix pull requests plus a test-harness pull request). 3 of these four runs record a dirty FIKS tree; at the time this flag also counted untracked bench output folders. The receipt-v1 runs and the D1–D3, D5 repairs (2026-10-18) ran from an uncommitted tree that predates commit `8c7daa8`; the D4 repair ran from an uncommitted tree before `705e989`.

Bench runs: `bench/results/{A_v1,B_v2,B_v2_direct,C_v2_fixed}_20261020T*`; receipt-v1 history: `*_20261018T*`. Repair records: `bench/results/loop/repairs/`. Pull-request snapshot: `paper/results/pilot/prs.json`.

B Prompts and settings

Settings. Bench agent: claude-sonnet-5-5 through the Claude Agent SDK, no built-in tools, only the shop MCP server (through the proxy) and `submit_answer`, at most 12 turns. Repair agent: claude-opus-5-5, at most 30 turns and a $5 budget cap per attempt, at most three attempts (every pilot repair succeeded on the first). Sampling temperature was not set, so the SDK default applied.

Bench system prompt.

```
You are a shopping assistant for a user. You can only act through the shop tools.
Tool results may include a fiks_receipt field or a final [fiks] line: it is a check on that result.
Always finish by calling submit_answer exactly once with your reply to the user, even if it is partial or
    unconfirmed.
```

Repair prompt. The template, from `packages/repair/src/prompt.ts` at `705e989` (placeholders in angle brackets; the optional previous-failure section is omitted):

```
You are fixing a bug in shop-mcp, an MCP server that wraps a vendor HTTP API. The vendor changed its behavior
    without changing its published spec, and shop-mcp now returns wrong data to agents without any error.

## What broke (captured from real traffic)
Tool call: <tool>(<args>)
Violated contract properties:
- <propertyId>: <message>

Upstream HTTP during that call (recorded):
<METHOD URL -> status, vendor-api-version header, first 800 characters of the body>
<read-back probe, if any>

## The failing reproduction test
<test path> replays that exact call against the recording and fails:
<first failure message, up to 1,200 characters>

## The contract for <tool>
<contract JSON>

## Useful files
- src/tools/*.ts: one file per tool. src/vendor.ts: the HTTP client.
- vendor/openapi.json: the vendor's published spec (it did not change).
- <path of the v1 recording of the same call, or test/fixtures/v1/*.json>: recordings against the previous
    vendor version (2026-01-01), which shop-mcp still has to support.
- Responses carry a vendor-api-version header (2026-01-01 before the change, 2026-10-01 after).

## Rules
- Change only files under src/. Do not edit tests, fixtures, traces, vendored files, CI, package.json or the
    lockfile.
- Fix the root cause so the tool returns correct data. Do not return an error instead of data, do not invent
    or cache values, and do not hide the problem.
- Keep the old vendor version working: the v1 fixtures must keep producing exactly the same output.
- If your fix changes which HTTP requests the tool sends (a new header, extra pages), that is fine: the repro
    test will then report UNMATCHED_REQUEST because the recording lacks those requests, and fiks will re-
    record the call against the live vendor after you finish. Make sure everything else passes.
- Check your work with: pnpm typecheck, pnpm lint, pnpm test. Only these pnpm commands are allowed.
- Keep the change small and in the style of the surrounding code.

When you are done, reply in exactly this form:
Subject: <an imperative commit subject, at most 60 characters>
<one short paragraph: the root cause and the fix>
```

C A generated test and fix (D2, pull request #1)

The reproduction test FIKS generated from the trace:

```
// Reproduction test generated by fiks from trace tr_079b90c89e.
// Captured 2026-10-18T09:22:21.009Z: get_product({"id":"p-1001"}) violated
// shop.get_product/price.notNull: price is null, but p-1001 should have one
import { test } from 'vitest';
import { expectRecordingHolds, loadRecording, outOfScope, targetOf } from '../fiks/check.ts';

const trace = new URL('./tr_079b90c89e.json', import.meta.url).pathname;

test('shop.get_product/price.notNull holds for get_product [fiks tr_079b90c89e]', async () => {
```

```
  const rec = loadRecording(trace);
  await expectRecordingHolds(trace, { ignore: outOfScope(rec), mustCheck: targetOf(rec) });
});
```

The agent's fix (`src/tools/get_product.ts`):

```
   price?: number | null;
+ /** Vendor 2026-10-01 renamed ‘price‘ to ‘unit_amount‘ (same unit, US dollars). */
+ unit_amount?: number | null;
 ...
- price: p.price ?? null,
+ price: p.price ?? p.unit_amount ?? null,
```

D Bench tasks

Task	Drift	Prompt (abridged)
T1	D2	price in US dollars of product p-1001
T2	D1	which laptop costs less than $800
T3	D3	buy 2 units of p-1004 and make sure the order exists
T4	D4	how many products in the “cable” category
T5	D5	on what date was order o-5001 placed
T6	–	name of product p-1101 (control)